
INFORMATION TECHNOLOGY UNIVERSITY (ITU), LAHORE

Alumni Engagement System

Project Scope & Metadata

Institution	Information Technology University (ITU), Lahore
Platform Type	Full-Stack Web Application
Document	Project Scope & Metadata
Supervisor	Dr. Muhammad Asif

Table of Contents

1. Project Scope	3
2. Student / Contributor Details	3
3. Project Supervisor Details	3
4. Specific Contributions	4
Frontend Contributions (User Interface & Experience)	4
Backend Contributions (Server, APIs & Database)	4
5. Appendix — Future Enhancements & Production-Readiness Roadmap	5
5.1 Features Planned for Completion	5
5.2 Security Hardening (Recommended Before Public Deployment).....	5

1. Project Scope

This document defines the boundaries of the Alumni Engagement System as delivered.

The delivered platform provides the following capabilities:

- **User accounts and roles** for three user types — Alumni, Sub-Administrator, and Main Administrator — with secure registration, login, and password hashing.
- **Alumni profile management**, including self-service editing of personal and academic details and password changes.
- **A searchable alumni directory** listing all registered members with their academic background.
- **Request-based private messaging** with real-time delivery and read tracking, including a full connection-request accept/reject workflow.
- **A community feed** for short public posts shared across all members.
- **A job board** supporting job posting, direct applications (with cover letter and resume link), applicant review, and application status management (Shortlisted / Accepted / Rejected).
- **An events hub** with event browsing, filtering, dedicated detail pages, event posting, and RSVP (with duplicate-RSVP prevention).
- **Administrative panels** for the Main Admin (Sub-Admin creation, alumni/sub-admin oversight, posting events and jobs) and Sub-Admins (event management, job posting, directory access, and a user-request handling area).
- **A structured, self-adapting user request form** for profile updates, account verification, certificate requests, bug reports, and account deletion requests.

2. Student / Contributor Details

Name	Asma Haider (BSSE Student)
GitHub	https://github.com/haider-devux
Email	asma.devux@gmail.com

3. Project Supervisor Details

Name & Title	Dr. Muhammad Asif (Postdoctoral – US, Assistant Professor)
Department	Department of Computer and Software Engineering, Faculty of Engineering
Institution	Information Technology University (ITU), Lahore, Punjab, Pakistan
Email	muhammad.asif@itu.edu.pk

4. Specific Contributions

The following frontend and backend contributions were completed by the contributor under the supervisor's guidance.

Frontend Contributions (User Interface & Experience)

- Built the complete **React application structure**, including the central navigation and page-routing system that connects all screens (home, login, dashboard, directory, events, jobs, posts, profile, and admin panels).
- Developed the **public landing/home page** presenting the ITU alumni network and its value proposition.
- Implemented the **registration and login experience**, including a single login screen with role selection (Alumni, Sub-Admin, Main Admin) and appropriate redirection to each role's dashboard.
- Created the **Alumni Dashboard** with a personalized welcome, quick-access feature cards, and a live event feed.
- Built the **Profile page** allowing members to view and update their personal, academic, and password details.
- Developed the **Alumni Directory** with live multi-field search and integrated per-member chat actions.
- Implemented the full **real-time chat interface** as a set of coordinated components — the floating chat window, active-conversation list, message thread with read status, incoming-request handling, and the directory's send-request control — all coordinated through a shared **chat context (global state)** that manages connections, requests, and live Socket.IO events.
- Built the **Community Feed (Posts)** for composing and browsing short updates.
- Developed the **Jobs module**, including the job board, the application form, and the applicant-review and status-update interface.
- Created the **Events experience** — event listings with filtering/search, event detail pages, RSVP handling, and an event-posting form component.
- Built the **Main Admin panel** (tabbed interface for Sub-Admin registration, event/job posting, and alumni/sub-admin directory), the **Sub-Admin panel**, and the **user-request handling** screen.
- Developed the **structured User Request Form** with dynamic, request-type-specific fields.
- Designed role-specific **navigation bars** (public, alumni, and sub-admin) and the shared visual styling of the application.

Backend Contributions (Server, APIs & Database)

- Set up the **Node.js / Express server** and configured the **PostgreSQL database connection** and cross-origin access for the frontend.
- Designed the **database schema**, including tables for registered alumni, sub-admins, events, event RSVPs, jobs, job applications, community posts, and the chat connection/message tables.
- Built the **authentication APIs** — alumni registration and login, sub-admin registration and login — with **bcrypt password hashing** and duplicate-email protection.

- Implemented the **profile-management API** for updating alumni details, including optional secure password changes.
- Developed the **community feed APIs** for creating and retrieving posts with author information.
- Built the complete **jobs API suite** — posting jobs, listing jobs, applying to jobs, retrieving applicants, and updating application statuses.
- Implemented the **events APIs** — creating events, listing events with organizer details, RSVP submission, and RSVP-status checks with duplicate prevention.
- Developed the **chat API and real-time layer** — connection requests, accept/reject handling, message retrieval and sending (with server-side verification that a connection is accepted), read-status updates, and the **Socket.IO real-time event handling** for instant delivery of requests, acceptances, and private messages.
- Created **directory/listing endpoints** for alumni and sub-admins to power the administrative panels, and a **database setup script** to provision the chat tables.

5. Appendix — Future Enhancements & Production-Readiness Roadmap

The following items are recognized, planned next steps. They are documented here transparently to define a clear path from the current working build toward a fully production-ready deployment. Their presence reflects a realistic, honest project scope rather than any gap in the core functionality delivered.

5.1 Features Planned for Completion

These modules are partially built as working prototypes and are slated to be fully wired to the live system:

- **Handle User Requests (Sub-Admin workflow):** Members can already *submit* structured requests. The administrator-side screen to *view, approve, and reject* those requests is currently a placeholder and will be connected to the backend to complete the support-ticket loop.
- **Advanced Event Management (Sub-Admin):** The dedicated “Manage Events” screen (with delete and attendee-tracking controls) currently operates on temporary in-page data. It will be connected to the live database so edits and deletions persist system-wide.
- **In-Widget Conversation View:** All messaging logic exists and messages are stored reliably; the detailed conversation thread will be fully surfaced inside the floating chat widget for a seamless in-place chat experience.

5.2 Security Hardening (Recommended Before Public Deployment)

The current build is a complete functional prototype. The following security upgrades are recommended before a live, public launch:

- **Session tokens (JWT):** Introduce proper server-issued login tokens so that user sessions are verified by the server on every request, rather than relying on the browser's local storage.
- **Server-side permission checks:** Add authorization on protected actions so the server confirms *who* is making each request (owner, sub-admin, or main admin) instead of trusting information sent by the browser.
- **Secured Main Admin access:** Replace the built-in/hard-coded Main Administrator credentials with a properly stored, hashed admin account managed like any other privileged user.
- **Gated Sub-Admin creation:** Restrict the sub-admin creation endpoint so that only an authenticated Main Admin can call it.